

EXERCISE 6 - Element-wise array multiplication with strip mining

Files : strip_serial.c, strip_parallel.c

(N = 1 000 000, C[i] = A[i] * B[i])

Compile / run

```
gcc -O2 -fopenmp strip_serial.c -o strip_serial && ./strip_serial
```

```
gcc -O2 -fopenmp strip_parallel.c -o strip_parallel && OMP_NUM_THREADS=4  
./strip_parallel
```

```
Exercise 06 - strip mining  
$ gcc -O2 -fopenmp strip_serial.c -o strip_serial  
$ gcc -O2 -fopenmp strip_parallel.c -o strip_parallel  
$ ./strip_serial  
Serial : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 time(200 reps) = 0.179000 s  
$ OMP_NUM_THREADS=1 ./strip_parallel  
Parallel : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 threads=1 strip=1024 time(200 reps) = 0.215000 s  
$ OMP_NUM_THREADS=2 ./strip_parallel  
Parallel : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 threads=2 strip=1024 time(200 reps) = 0.168000 s  
$ OMP_NUM_THREADS=4 ./strip_parallel  
Parallel : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 threads=4 strip=1024 time(200 reps) = 0.146000 s  
$ OMP_NUM_THREADS=8 ./strip_parallel  
Parallel : C[10]=10.0 C[N-1]=999999.0 checksum=499999500000.0 threads=8 strip=1024 time(200 reps) = 0.196000 s
```

Strip mining

The single loop 0..N-1 is split into two loops:

```
#define STRIP 1024           // multiple of the SIMD width  
  
#pragma omp parallel for schedule(static) // strips shared between threads  
for (int i = 0; i < N; i += STRIP) {  
    int end = (i + STRIP < N) ? i + STRIP : N; // clamp the last strip  
    #pragma omp simd           // strip itself is vectorised  
    for (int j = i; j < end; j++)  
        C[j] = A[j] * B[j];  
}
```

- * OUTER loop walks the array one strip at a time - OpenMP distributes these strips across the threads.
- * INNER loop processes one strip - a short, fixed-length loop the compiler turns into SIMD instructions.
- * Strip size 1024 floats: a multiple of both the AVX2 width (8 floats) and the AVX-512 width (16 floats), so every strip maps onto whole vector registers; 4 KB per strip also fits comfortably in L1 cache.

Results

Checksum is identical in every run (499999500000). The kernel is repeated 200x to get a measurable time. Speed-up is small because a 1-FLOP-per-element multiply is limited by MEMORY BANDWIDTH, not by compute - a couple of threads already saturate it.